

Deep-SmallThinker-3B Benchmark Report

SmallThinker-3B-Preview (Q4_K_M, 2.1 GB) on NVIDIA Jetson Orin Nano 8GB | llama.cpp CUDA | 10 prompts (5 general + 5 coding) | temp 1.0, 32K context, 8192 thinking tokens | GUI off

Key Finding

Deep-thinking settings (temp 1.0 + excess thinking tokens) do **not** raise average quality versus the earlier temp-0.3 run: **6.4/10 both ways**. They **shift** where quality lands. Structured tasks with a verifiable answer (code, math, coding) reach **8/10** — the extra step-by-step reasoning helps. But open-ended creative/prose tasks **collapse to 3-4/10**: the model enters a meta-reasoning loop ("I need to write X... first I should... let me think...") and never emits the actual deliverable, burning the whole 8192-token budget on thinking-about-thinking.

1. Speed, Metrics & Quality (all 10 prompts)

Generation speed is rock-steady at ~20 tok/s (memory-bandwidth bound). Prompt eval averages ~409 tok/s. Tok* is estimated from characters/4 (this llama.cpp build does not print n_eval).

Prompt	Suite	Gen tok/s	Ppt tok/s	Tok*	Wall sec	Think %	Qual 1-10
code	general	19.9	379	2057	107.1	15%	8
iambic	general	20.0	321	737	36.9	91%	4
prose	general	20.1	426	987	43.8	100%	7
creative	general	20.1	382	1894	85.4	100%	3
math	general	20.0	350	693	40.6	98%	8
html	coding	20.1	422	1932	98.9	73%	7
python	coding	20.0	445	3855	168.7	3%	8
c	coding	19.9	428	5281	253.4	15%	8
basic	coding	20.1	533	1674	85.8	92%	4
julia	coding	20.1	406	2284	118.9	38%	7
AVERAGE		20.0	409				6.4

Quality splits cleanly by task type. Every verifiable/coding task scores 7-8/10. Every open-ended creative-form task (iambic, creative, basic) collapses into a thinking loop and scores 3-4/10.

2. Thinking vs Answer — the Deep-Thinking Signature

This is the metric that a normal benchmark misses. "Think %" is the share of output spent in chain-of-thought reasoning before the final answer. A healthy thinking model lands at 10-40% (reason, then answer). At >80% it has fallen into a meta-reasoning loop and produced no deliverable.

Prompt	Thinking chars	Answer chars	Think ratio	Outcome
code	1210	7017	15%	Clean answer (7.0K chars code)
iambic	2687	261	91%	LOOP — budget burned, poem truncated
prose	3947	0	100%	Meta-reasoning, no clean answer block
creative	7578	0	100%	LOOP — 7.6K chars of 'I need to write...'
math	2710	62	98%	Clean proof (62-char boxed answer)
html	5676	2053	73%	Clean page (2.0K chars)
python	522	14899	3%	Clean code (14.9K chars)
c	3197	17928	15%	Clean code (17.9K chars)
basic	6193	501	92%	LOOP — program truncated (501 chars)
julia	3507	5630	38%	Clean code (5.6K chars)

The three failures (iambic 91%, prose 100%, creative 100%, basic 93%) are all open-ended creative-form tasks. SmallThinker's training data is dominated by verifiable math/code reasoning, so on unverifiable creative prompts it keeps re-planning the task instead of committing to an answer. This is the central weakness of reasoning-first small models at high temperature.

3. Settings Under Test

Setting	Value	Rationale
temperature	1.0	High temp favors reasoning-tree exploration — tokens/time are free on the Jetson
context	32768	Native 32K — GUI off frees enough RAM for full context
n_tokens (thinking)	8192	Excess thinking budget — model may reason as long as it wants on one turn
top_p / top_k	0.95 / 40	Standard nucleus sampling
repeat_penalty	1.1	Discourages reasoning-loop repetition
--jinja	on	Critical — without it the model loops on ~49 internal tokens
quantization	Q4_K_M	2.1 GB — smaller weights buy the 16K-32K contexts

4. Recommendations

Guidance	Rationale
Use deep thinking (temp 1.0, excess tokens)	for math proofs, algorithms, and code — it reaches 8/10 with full step-by-step reasoning.
Use a lower temperature (temp 0.6) or fewer tokens	for creative writing, poetry, and prose — high temp + excess tokens trigger meta-reasoning loops.
Treat the chain-of-thought as a draft	SmallThinker is a 3B model; its reasoning is often confident but can be wrong (it failed the classic bat-and-ball trick earlier).

Guidance	Rationale
Keep --jinja mandatory	without it the model never escapes its internal thinking and emits ~49 tokens only.

Methodology: identical prompt text to the jetson-llm-benchmark multiprompt_bench.py (general 5) and bench_coding.py (coding 5) suites. Quality scored 1-10 by the same rubric. Tokens estimated from chars/4.